

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

EIN092B: Visualización

Técnicas de evaluación de algoritmos de clasificación y regresión

Sesión 9

Jorge Portilla

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

Tabla de Contenidos

1 Técnicas de evaluación de algoritmos de clasificación

2 SHAP Library

3 Validación cruzada

Técnicas de evaluación de algoritmos de clasificación

Evaluación de Algoritmos

Groundtruth

Información pre-definida que representa el resultado esperado para la resolución de cierto problema.

- Esta información se utiliza como base de comparación cuando se evalúan algoritmos para la resolución del mismo problema.
- En imágenes y video, cuando se mide la habilidad de cierto algoritmo para enfrentar cierto problema, en general, se utilizan métricas que puedan mostrar:
  - **Un valor definido** (e.g., error de localización 2D, intervalo de tiempo entre la aparición del objeto en el ground-truth y su detección en el sistema, ...). En este caso, se pueden obtener estadísticos de estos valores (media, desviación estándar, mínimo y máximo, ...) o ver su evolución en el tiempo.

Evaluación de Algoritmos

- . . . en general, se utilizan métricas que puedan mostrar:
  - **Un número de éxitos o fallas del algoritmo** (proviene de clasificación binaria). En este caso, se usan las siguientes definiciones:
    - **Verdadero Positivo (TP):** El algoritmo detecta una situación real (existe en la ground-truth y en los resultados del algoritmo).
    - **Falso Negativo (FN):** No se detectó una situación real (existe sólo en la ground-truth).
    - **Falso Positivo (FP):** El algoritmo detecta una situación irreal (existe sólo en los resultados del algoritmo).
    - **Verdadero Negativo (TN):** Elemento que no aparece ni en la ground-truth ni en los resultados del algoritmo.

Evaluación de Algoritmos

- TP, FP, TN, FN también pueden dar más información para caracterizar a los algoritmos:
  - **Precision** $= \frac{TP}{TP + FP}$, tasa de aciertos, contra aciertos y errores del algoritmo[1].
  - **Sensitivity** $= \frac{TP}{TP + FN}$, tasa de aciertos, contra todo lo que se debía detectar[2].
  - **Specificity** $= \frac{TN}{FP + TN}$, tasa de elementos correctamente no detectados, contra todo lo que no debía generar respuesta.
  - **F-score** $= 2 * \frac{\text{Precision} * \text{Sensitivity}}{\text{Precision} + \text{Sensitivity}}$, media armónica entre Precision y Sensitivity.
  - **Acc**: $= \frac{TN + TN}{FP + FN + TN + TP}$

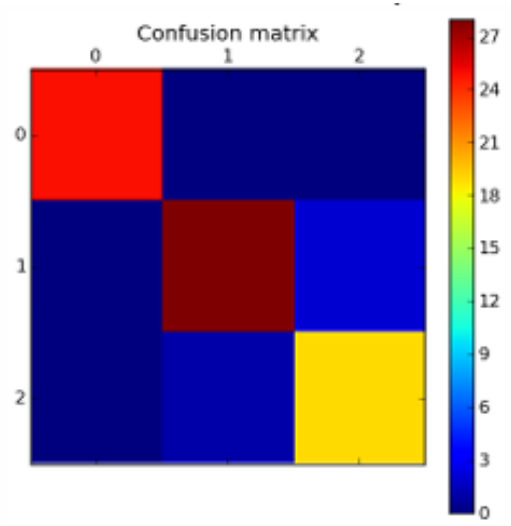
[1] El objetivo es medir la capacidad del modelo para no identificar erróneamente una señal negativa como positiva.

[2] Este criterio mide la capacidad del modelo para encontrar todas las señales positivas en el conjunto de datos.

Evaluación: Matriz de Confusión (Confusion Matrix)

- Es una herramienta de visualización típicamente usada en aprendizaje supervisado (muestras de entrenamiento etiquetadas).
- Cada columna de la matriz representa las instancias de clases predichas (e.g., la salida de un algoritmo).
- Cada fila representa las verdaderas instancias de la clase (e.g., etiquetas de las muestras, conocidas a priori).
- También, se encuentra la misma representación matricial transpuesta (en este caso, normalmente se aclara).

		prediction outcome		
		p	***n***	**total**
actual value	***p'***	True Positive	False Negative	P'
	n'	False Positive	True Negative	N'
	total	P	N	



Matriz de Confusión - Ventajas

- Es fácil ver si el sistema se confunde entre dos o más clases.
- Cuando un conjunto de datos es desbalanceado (número de muestras en diferentes clases varía mucho) la tasa de error del clasificador no es representativa del verdadero rendimiento del clasificador.
- *e.g.* clase A: 990 muestras - clase B: 10 muestras: **¿Qué pasa con la tasa de error si acierto en todas con A y en ninguna con B?**

Curvas ROC

- La curva ROC (Receiver Operating Characteristic) es una gráfica que ilustra el rendimiento de un clasificador binario al variar su umbral.
- Se crea una gráfica de la **TPR** (true positive rate) frente a la **FPR** (false positive rate) para distintos umbrales.

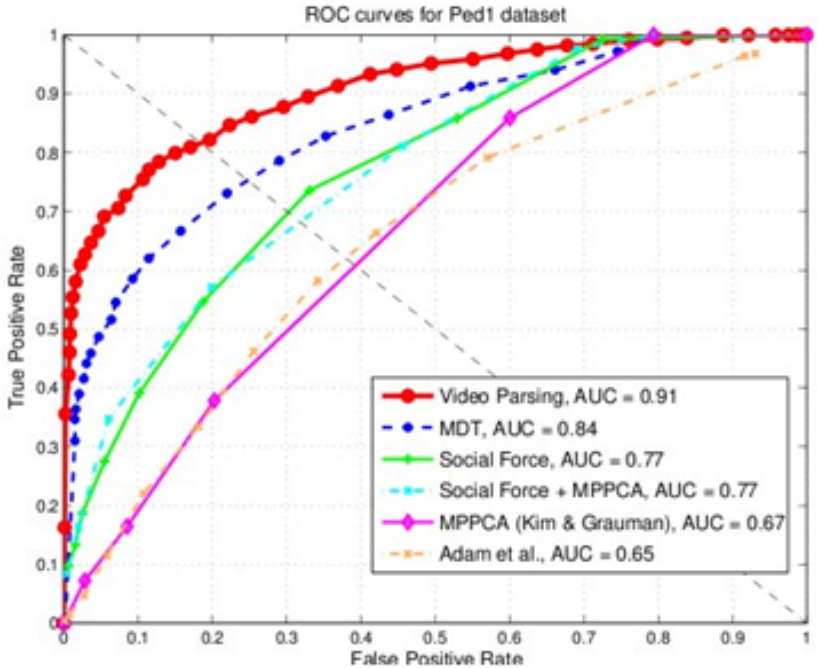
- **TPR (True Positive Rate):**

$$\mathbf{TPR} = \frac{\mathbf{TP}}{\mathbf{TP} + \mathbf{FN}} = \mathbf{Sensibilidad}$$

- **FPR (False Positive Rate):**

$$\mathbf{FPR} = \frac{\mathbf{FP}}{\mathbf{FP} + \mathbf{TN}} = 1 - \mathbf{Especificidad}$$

Curvas ROC



Métricas

- **TPR (True Positive Rate)**:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \text{Sensibilidad}$$

- **FPR (False Positive Rate)**:

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = 1 - \text{Especificidad}$$

Construcción de la Curva ROC

Paso 1: Definir los datos

Supongamos que tenemos un conjunto de etiquetas reales y predicciones probabilísticas del modelo.

Muestra	Etiqueta Real	Predicción
1	1	0.95
2	0	0.85
3	1	0.80
4	0	0.70
5	1	0.65
6	0	0.60
7	1	0.55
8	0	0.30
9	1	0.20
10	0	0.10

Tabla 1: Datos del modelo: etiquetas reales y predicciones.

Construcción de la Curva ROC

Paso 2: Definir los umbrales

Los umbrales serán los valores únicos de las probabilidades predichas. Los umbrales son:

$$\text{Umbrales} = [0{,}95, 0{,}85, 0{,}80, 0{,}70, 0{,}65, 0{,}60, 0{,}55, 0{,}30, 0{,}20, 0{,}10]$$

Paso 3: Calcular TPR y FPR para cada umbral

Usamos las siguientes fórmulas:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$$

Construcción de Curva ROC

La tabla siguiente resume los cálculos para cada umbral:

Umbral	TP	FP	FN	TN	TPR	FPR
0.95	1	0	4	5	0.20	0.00
0.85	1	1	4	4	0.20	0.20
0.80	2	1	3	4	0.40	0.20
0.70	2	2	3	3	0.40	0.40
0.65	3	2	2	3	0.60	0.40
0.60	3	3	2	2	0.60	0.60
0.55	4	3	1	2	0.80	0.60
0.30	4	4	1	1	0.80	0.80
0.20	5	4	0	1	1.00	0.80
0.10	5	5	0	0	1.00	1.00

Tabla 2: Cálculos de TPR y FPR para cada umbral.

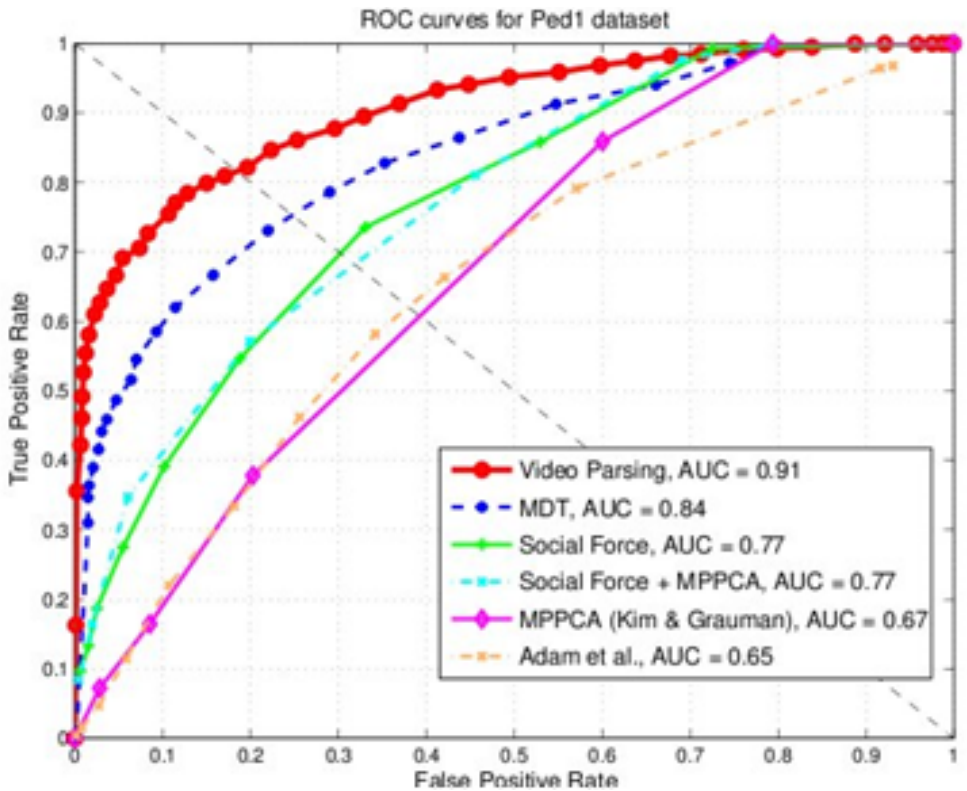
Construcción de Curva ROC

Paso 4: Graficar la curva ROC

Con los valores de TPR y FPR calculados, podemos trazar la curva ROC. Los puntos clave para la curva son:

$$\text{FPR} = [0{,}00, 0{,}20, 0{,}20, 0{,}40, 0{,}40, 0{,}60, 0{,}60, 0{,}80, 0{,}80, 1{,}00]$$

$$\text{TPR} = [0{,}20, 0{,}20, 0{,}40, 0{,}40, 0{,}60, 0{,}60, 0{,}80, 0{,}80, 1{,}00, 1{,}00]$$



Grid Search

- **Grid Search** es una técnica de optimización utilizada para encontrar la combinación óptima de hiperparámetros en un modelo de ML.
- Se define una malla de posibles valores para cada hiperparámetro.
- El modelo se entrena y se evalúa exhaustivamente con cada combinación posible de hiperparámetros en la malla.

Pasos del Grid Search

1. **Definir los hiperparámetros**: Seleccionar los hiperparámetros del modelo que se desean ajustar.
2. **Establecer el rango de valores**: Especificar los valores posibles de cada hiperparámetro.
3. Entrenar el modelo con cada combinación de hiperparámetros y evaluarlo mediante un método como la validación cruzada.
4. **Seleccionar la mejor combinación**: Escoger la combinación de hiperparámetros que ofrezca el mejor rendimiento según la métrica de evaluación seleccionada.

GridSearchCV

- Definir un modelo de machine learning. Este modelo se entrenará múltiples veces con diferentes combinaciones de hiperparámetros.

- GridSearchCV entrena el modelo con todas las combinaciones de los hiperparámetros que definiste, evaluando cada una mediante validación cruzada y almacenando los resultados.

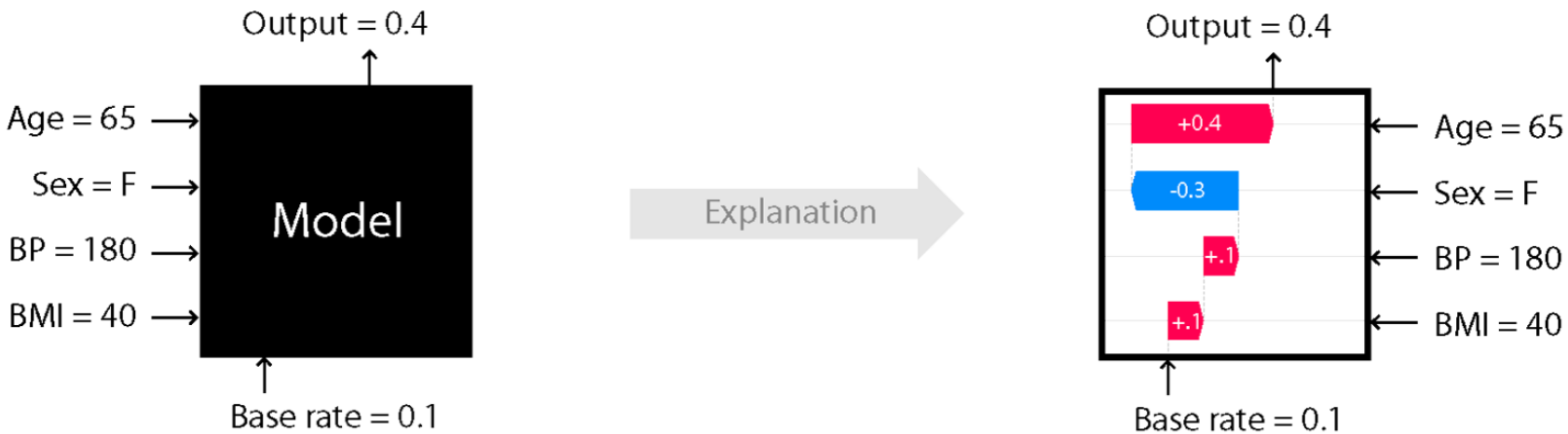
- Selección del mejor modelo: Después de probar todas las combinaciones, selecciona el que dio los mejores resultados según la métrica de evaluación definida (como accuracy, F-score, RMSE, entre otras).

SHAP Library

SHapley Additive exPlanations



SHAP



SHAP

Para cada predicción, SHAP asigna a cada variable un valor ϕ_i que indica cuánto movió la salida del modelo respecto al promedio:

$$f(x) = \mathbb{E}[f(X)] + \sum_i \phi_i$$

- $\phi_i > 0$: la variable aumenta la predicción.
- $\phi_i < 0$: la baja.
- Las contribuciones suman exactamente la predicción.

Ejemplo

$$0{,}1 + 0{,}4\,(\text{Age}) - 0{,}3\,(\text{Sex}) + 0{,}1\,(\text{BP}) + 0{,}1\,(\text{BMI}) = 0{,}4$$

Uso básico

```python
import shap, xgboost

X, y = shap.datasets.california()
model = xgboost.XGBRegressor().fit(X, y)

explainer   = shap.Explainer(model, X)  # elige el algoritmo segun el modelo
shap_values = explainer(X)

shap.plots.waterfall(shap_values[0])    # una muestra
shap.plots.beeswarm(shap_values)        # todo el dataset
shap.plots.bar(shap_values)             # importancia global
```

Funciona con árboles (rápido y exacto), modelos lineales y redes neuronales. Para modelos de caja negra usa `KernelExplainer`, que es bastante más lento.

Precauciones

- SHAP explica el modelo, no el fenómeno: no implica causalidad.
- Si el modelo generaliza mal, la explicación tampoco sirve. Primero, validar (k-fold, ROC); después, explicar.
- Con variables muy correlacionadas, la importancia se reparte entre ellas.

Ejemplo

It uses an XGBoost model trained on the classic UCI adult income dataset (which is a classification task to predict if people made over $50k in the 90s).

```
import xgboost
import shap

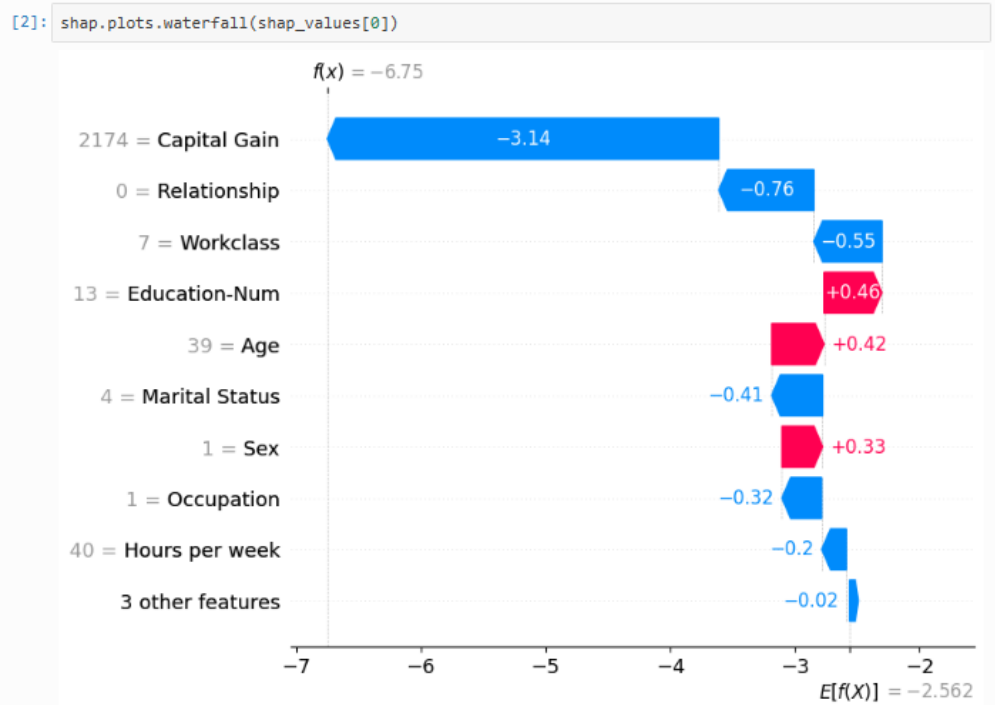
# train XGBoost model
X, y = shap.datasets.adult(n_points=2000)
model = xgboost.XGBClassifier().fit(X, y)

# compute SHAP values
explainer = shap.Explainer(model, X)
shap_values = explainer(X)

```

shap.plots.waterfall

Display explanations for individual predictions



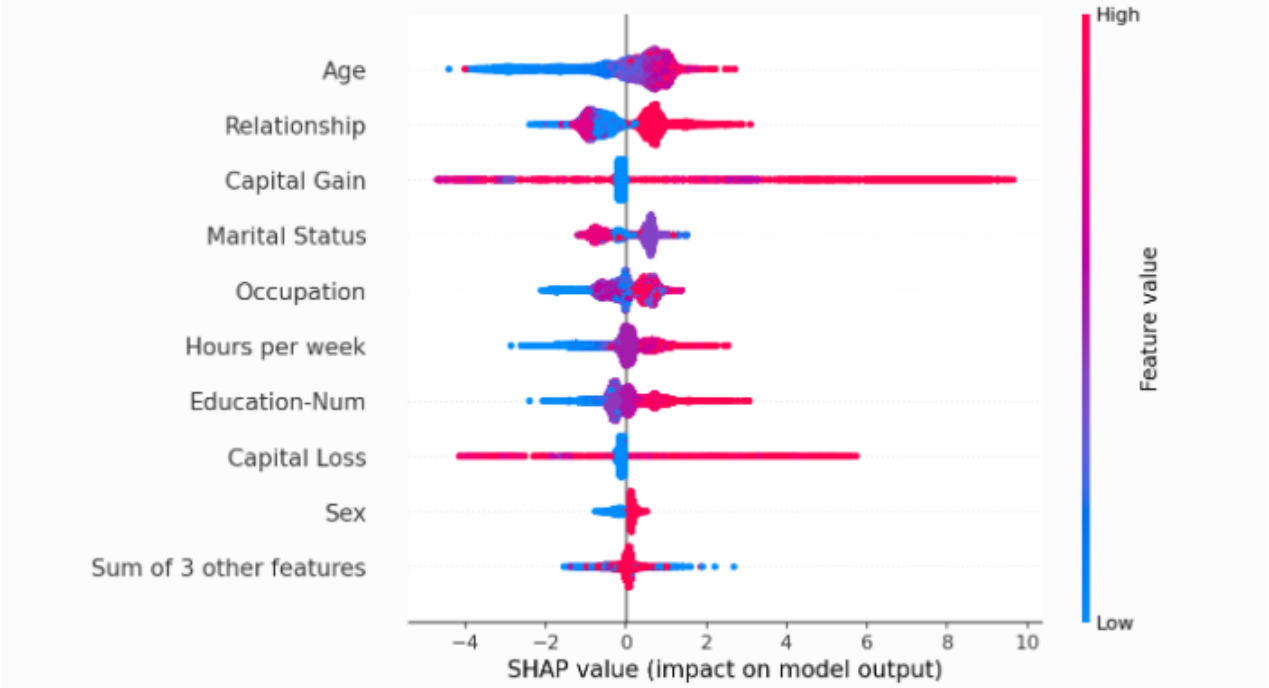
- La parte inferior del gráfico comienza en el valor esperado de la salida del modelo[3]
- Parte del valor base y termina con la predicción de esa muestra.
- Barras rojas suman, azules restan.[4]

[3] $E[f(X)]$ es el valor base: la predicción promedio del modelo sobre el conjunto de referencia. $f(x)$ es la predicción para la muestra analizada.

[4] Se evalúa el modelo con Capital Gain = 2174 y las demás variables en sus valores reales. Se evalúa otra vez, ahora con Capital Gain tomada de muestras del conjunto de referencia (por ejemplo 30, 42, 58...)

shap.plots.beeswarm

```
[2]: shap.plots.beeswarm(shap_values)
```



- Cada punto es una muestra; el eje x indica su valor de SHAP.
- Color: valor de la variable (rojo: alto, azul: bajo).
- Variables ordenadas de mayor a menor influencia.
- Rojo a la derecha: valores altos aumentan la predicción.

Validación cruzada

¿Cómo seleccionar un modelo en la práctica?

- ¿Cómo elegir automáticamente los parámetros del modelo?
- Asumiremos que tenemos un conjunto finito de modelos $\mathcal{M} = \{M_1, \ldots, M_d\}$.
- En caso de que el(los) parámetro(s) sea(n) continuo(s), podemos discretizarlo(s).
- Como mencionamos antes, elegir los parámetros que minimizan el error de entrenamiento no garantiza la generalización.

Cross-Validation en Aprendizaje Automático

- **Problema con conjuntos de entrenamiento y prueba fijos**:
  - Un conjunto de prueba pequeño implica una alta incertidumbre estadística.
  - Dificultad para comparar algoritmos debido a la variabilidad del error estimado.
- **Datasets extensos**, no presentan este problema.
- Permiten utilizar todos los ejemplos para estimar el error medio de prueba. Implican un mayor costo computacional.

Cross-Validation en Aprendizaje Automático

- **Problema con conjuntos de entrenamiento y prueba fijos**:
  - Un conjunto de prueba pequeño implica una alta incertidumbre estadística.
  - Dificultad para comparar algoritmos debido a la variabilidad del error estimado.
- **Datasets extensos**, no presentan este problema.
- Permiten utilizar todos los ejemplos para estimar el error medio de prueba. Implican un mayor costo computacional.
- Uno de los métodos más conocidos es **k-Fold Cross-Validation**, que divide el conjunto de datos en k subconjuntos no superpuestos. Estimar el error de prueba promedio a partir de los k ensayos.
- **Consideraciones**:
  - No existen estimadores insesgados de la varianza de estos estimadores de error promedio.
  - Se utilizan aproximaciones para estimar la varianza (Bengio y Grandvalet, 2004).

Validación cruzada con conjunto de prueba (Hold-out)

- Una opción es hacer lo siguiente:

Algoritmo: Validación cruzada con conjunto de prueba

1. Dividir aleatoriamente S en S_{train} (por ejemplo, el 75 % de los datos) y S_{cv} (el 25 % restante). Donde S_{cv} es el conjunto de validación cruzada.
2. Entrenar cada modelo M_i en S_{train} solo para obtener la hipótesis f_i.
3. Seleccionar y mostrar la hipótesis f_i con el menor error $\hat{\epsilon}_{S_{\text{CV}}}(f_i)$ en el conjunto de validación cruzada.

Validación cruzada con conjunto de prueba (Hold-out)

- Una opción es hacer lo siguiente:

Algoritmo: Validación cruzada con conjunto de prueba

1. Dividir aleatoriamente S en S_{train} (por ejemplo, el 75 % de los datos) y S_{cv} (el 25 % restante). Donde S_{cv} es el conjunto de validación cruzada.
2. Entrenar cada modelo M_i en S_{train} solo para obtener la hipótesis f_i.
3. Seleccionar y mostrar la hipótesis f_i con el menor error $\hat{\epsilon}_{S_{\text{CV}}}(f_i)$ en el conjunto de validación cruzada.

- Así, para cada f_i obtenemos una estimación más precisa del error de generalización.
- Y podemos seleccionar la hipótesis con el error de generalización estimado mínimo.
- Usualmente, el tamaño del conjunto de validación cruzada se establece entre 1/4 y 1/3.
- Estamos perdiendo datos que no usamos en el entrenamiento. Y estimamos el error de generalización únicamente a partir de un conjunto de validación cruzada.

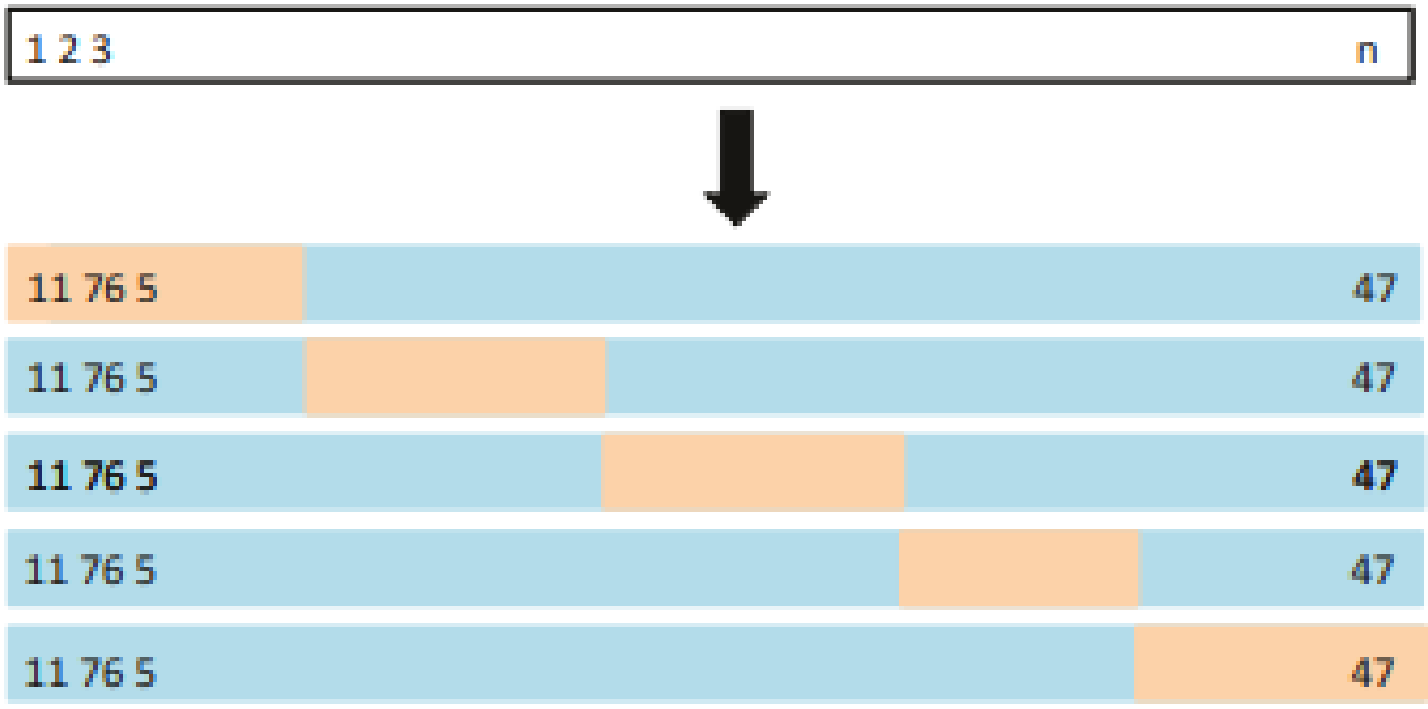
Validación Cruzada K-Fold

Algoritmo: Validación Cruzada K-Fold

1. Dividir aleatoriamente S en K subconjuntos de M/K ejemplos de entrenamiento cada uno. Llamemos a estos subconjuntos $S_1, \ldots, S_K$.
2. Para cada modelo M_i hacer:
   1) Para $j = 1, \ldots, K$ hacer:
      1) $f_{ij} \leftarrow A_i(S \setminus S_j)$ (entrenar con todos los datos excepto S_j)
      2) $\hat{\epsilon}_{S_j}(f_{ij}) = \frac{1}{|S_j|} \sum_{(x_m, y_m) \in S_j} \ell(y_m, f_{ij}(x_m))$ (calcular el error sobre el conjunto de validación)
   2) $\hat{\epsilon}_{M_i} = \frac{1}{K} \sum_{k=1}^{K} \hat{\epsilon}_{S_k}(f_{ik})$ (calcular el promedio del error de validación sobre los k pliegues)
3. Seleccionar el modelo M_i con el menor $\hat{\epsilon}_{M_i}$.
4. $f \leftarrow A_i(S)$ (entrenar una hipótesis con todos los datos según el modelo M_i).
5. Salida: f

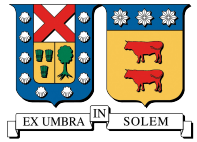
- A_i es el algoritmo de aprendizaje según el modelo M_i.

k-fold cross validation



Validación Cruzada K-Fold (2)

- Usualmente $K = 5$ o $K = 10$.
- Cuando $K = M$ este método se llama validación cruzada leave-one-out.
- En problemas de clasificación, a veces la proporción de ejemplos de cada clase es desigual. En este caso, podemos adaptar la validación cruzada de modo que cada pliegue mantenga las mismas proporciones. Así, todos los pliegues estarán igualmente desequilibrados.
- **Este método se llama validación cruzada K-Fold estratificada.**



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

EIN092B: Visualización

Técnicas de evaluación de algoritmos de clasificación y regresión

Sesión 9

Jorge Portilla

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile